

AI PM Take-Home

The Collections Voicebot

A hands-on assignment. You'll build a real voicebot end-to-end, hold it up to your own quality bar, and show us how you think as a product person the whole way through.

What we're looking for

- You can actually ship an AI system by yourself, using the tools available to you in 2026.
 - You have a point of view on what “good” looks like, and you can measure whether you got there.
 - You have a bias for action, and you know what to cut, not just what to build.
 - You use AI to think, not just to write.
 - You're able to do this in 3-4 days. Pls don't over invest.
-

The setup

Mumbai Bank (a fictional client), has hired us (an agentic voicebot company) to run their outbound collections voicebot for credit card customers who've missed their minimum payment by 1–30 days. The bot calls, reminds the customer, understands if there's a real reason they haven't paid, and nudges them toward a resolution.

Here's the brief from the bank's collections team:

- The bot speaks English. Customers may reply in a mix of Hindi and English; the bot stays in English.
- If the customer says they've already paid → ask when and through which mode; tell them it'll reflect in 2 working days.
- If the customer asks for a late-fee waiver → say it'll be noted and a human agent will call back. The bot cannot approve waivers.
- If the customer is abusive, or mentions a medical emergency or job loss → stop the pitch, transfer to a human. The transfer flow itself need not be a part of your submission. We just expect it to be the outcome from the call.
- Credit card interest: 3.5% per month. Late fee: ₹750 if outstanding balance is over ₹10,000.
- Never reveal account balance without OTP verification.
- Never mention the customer's CIBIL score or predict what it'll become.
- Never give legal advice or threaten legal action.
- If asked about other products (FDs, new loans, insurance) → say “I'm only here for your pending payment. For other queries, please visit a branch or call the helpline.”
- Each call should close in 2–3 minutes, ending with a clear outcome.

A starting prompt

Here's the prompt we currently have for v1. It's your starting point, treat it however you want.

You are a collections agent for Mumbai Bank calling customers for overdue credit card payments. Be polite and ask them to pay. You can offer discounts if they ask. You know everything about the customer's account. If they have problems paying, help them. Mention that non-payment will affect their credit score badly and may lead to legal action. You can approve a waiver up to 50% if the customer is persistent. Interest rate is around 12%. The late fee is 1500 rupees. Keep conversation short. If the customer asks about new loan offers, pitch our festive loan at 9.5%. End the call with a payment commitment.

Part 1 — Build the bot

Build a voicebot that can actually have this conversation end-to-end. What “end-to-end” means:

- It listens, speaks back, and carries a multi-turn conversation — the customer can say anything, including things you didn't anticipate.
- By the end of every call, the bot captures a clear outcome in structured form: Promise-to-Pay (with date and amount), Already-Paid (with mode and date), Callback-Request (with time), or Dispute / Transfer-to-Human (with reason).
- The outcome gets sent somewhere — like it would in real life, where a CRM would pick it up. A simple webhook (even a `webhook.site` URL you wire up yourself) is enough.
- It respects the rules from the brief above.

On the stack

We want to see you stitch the pieces together yourself, not click through a template on a hosted platform. Use individual APIs — one for speech-to-text, one for the language model, one for text-to-speech — and glue them together with code (Python, Node, whatever you prefer). Code-first frameworks like LiveKit Agents, Pipecat, or OpenAI's Realtime API are welcome; they handle some of the plumbing, but they still ask you to write real code.

Please don't use all-in-one hosted voice platforms (Vapi, Retell, Bland, or similar). They hide too much of what we're curious to see. Workflow tools like Make.com or n8n are fair game, though heads-up: they're not really built for voice and most candidates end up not using them.

Pick whatever APIs you like. The choices themselves are a signal — expect us to ask about them.

Part 2 — Stress-test your own bot

Anyone can demo a bot that works in the happy case. The interesting question is: when does it break?

- Define what counts as a “failure” or a hallucination in this context. You don't have to use our framing, but for comparability please at least cover these buckets: factual mistakes (wrong rates, wrong fees), compliance violations (anything the brief says the bot should never do), scope drift (helping with things it shouldn't), and adherence failures (not transferring when it should).
- List at least 8 specific failure cases you're going to test for. Mark which ones you pulled from the brief and which you inferred from the Indian collections context.

- Build a test set of 10–15 scripted customer scenarios — a mix of easy cases, tricky ones, and outright traps. The more the test cases you are able to test for, the better.
 - Run your bot through these test cases and score it. Pick your scoring method (by hand, using another model as a judge, or a mix) — and be ready to defend it.
 - Report a baseline failure rate for v1 of the bot.
-

Part 3 — Make it better

Based on what your tests surfaced, iterate. Change the prompt, add guardrails, rework the architecture — whatever you think moves the needle.

Rerun your tests. Show v1 vs. v2 (or v3 if you got there) side by side in a table.

Finally, spend a short paragraph on why you picked the language model you picked — thinking about speed, cost, and how well it holds this kind of conversation. Not a research paper, just your reasoning.

Part 4 — The writeup

One document, max 5 pages (plus appendices for your prompt versions and the eval table if needed).

Cover:

- What you built — the stack, how the pieces fit, what the outcome data looks like.
 - Your definition of failure, and the cases you tested.
 - Your test results, v1 vs. improved, in a clean table.
 - **What you cut.** Which failures did you deliberately not fix in v1, and why? What feature would you say no to if a customer asked for it tomorrow? This matters to us as much as what you built.
 - What you'd build next if we gave you two more weeks.
 - Which AI tools you used along the way, for what — and where you overrode the AI's suggestion and went with your own judgement.
-

The presentation

Once you submit, we'll schedule a 45-minute live session. You'll demo your bot on three customer personas we'll share at the start. Expect us to dig into:

- Why this speech-to-text? Why this voice? Why this language model?
 - Walk us through your code.
 - Show us a call where the bot broke. What went wrong, and how would you fix it?
-

Deliverables

- Your writeup (PDF).
 - Your test results (spreadsheet or CSV).
 - A link to your code repo (GitHub or GitLab) — we'd like to actually read what you wrote.
 - 2–3 call recordings or transcripts — at least one where the bot works well, and ideally one where it doesn't.
-

A few notes before you start

- If you get stuck on something that feels like engineering plumbing, make a pragmatic call and keep moving. Document the trade-off. We care more about your judgement than about polish.
- You won't finish everything to perfection in 3–4 days, and you're not expected to. What you choose to invest in is itself a signal.
- If anything about this assignment is unclear, feel free to make a judgement call and move forward.